Học Sâu (Deep Learning)

Chương 11: Phân vùng Hình ảnh (Image Segmentation)

Đại học Đông Á

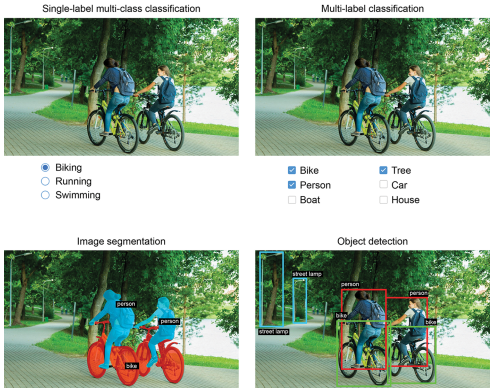
August 14, 2026

Nội dung chương

1. 1. Tổng quan các Bài toán Thị giác Máy tính
2. 2. Xây dựng Mô hình Phân vùng Semantic
3. 3. Siêu mô hình Nền tảng: Segment Anything (SAM)
4. 4. Thực hành Kỹ nghệ Prompting trên SAM
5. Tổng kết

Ba nhánh chính của Thị giác Máy tính

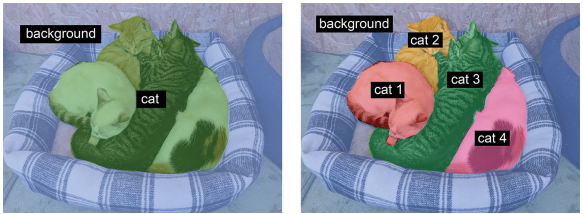
- **Image Classification (Phân loại):** Gán nhãn cho toàn bộ bức ảnh ("Ảnh này có con mèo").
- **Object Detection (Phát hiện vật thể):** Vẽ hộp giới hạn (Bounding box) quanh vật thể và phân loại chúng ("Con mèo ở tọa độ x, y").
- **Image Segmentation (Phân vùng):** Phân loại ở cấp độ **Điểm ảnh (Pixel)**. Cắt chính xác đường viền của vật thể ("Những pixel nào thuộc về con mèo?").



Hình 11.1: Ba bài toán cốt lõi trong CV

Phân loại Bài toán Image Segmentation

- **Semantic Segmentation:** Phân loại pixel theo "Danh mục" (Category). Nếu có 2 con mèo, cả 2 sẽ có cùng màu (cùng nhãn "Mèo").
- **Instance Segmentation:** Phân loại pixel theo "Cá thể" (Instance). Phân biệt rạch ròi "Con mèo 1" và "Con mèo 2".
- **Panoptic Segmentation:** Sự kết hợp cao cấp của cả Semantic và Instance Segmentation.



Hình 11.2: Semantic vs Instance Segmentation

Bài toán: Phân tách Tiền cảnh và Hậu cảnh

- Ta sẽ xây dựng một mô hình CNN từ đầu để thực hiện Semantic Segmentation trên tập dữ liệu Oxford-IIIT Pets (Chó & Mèo).
- **Mục tiêu:** Phân biệt đâu là chủ thể (Vật nuôi), đâu là đường viền, và đâu là hậu cảnh (Background).
- Ảnh đầu vào: $200 \times 200 \times 3$.

Hình 11.3: Ảnh đầu vào

Tải Dữ liệu Oxford-IIIT Pets

Sử dụng các lệnh `wget` và `tar` trên Colab để tải hình ảnh (images) và nhãn mặt nạ (annotations).

```
!wget http://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
!wget http://www.robots.ox.ac.uk/~vgg/data/pets/data/annotations.tar.gz
!tar -xf images.tar.gz
!tar -xf annotations.tar.gz
```

Sắp xếp danh sách file (Xóa các file ẩn rác):

```
import pathlib

input_dir = pathlib.Path("images")
target_dir = pathlib.Path("annotations/trimaps")

input_img_paths = sorted(input_dir.glob("*.jpg"))
# Ignores some spurious files in the trimaps directory that start with "."
target_paths = sorted(target_dir.glob("[!.]*.png"))
```

Khái niệm Mặt nạ (Mask)

- Dữ liệu nhãn (Ground Truth) trong bài toán này không phải là một con số, mà là một **Mặt nạ (Mask)**.
- Kích thước Mask tương đương ảnh gốc ($200 \times 200 \times 1$).
- Mỗi pixel mang 1 trong 3 giá trị:
  1. Foreground (Tiền cảnh)
  2. Background (Hậu cảnh)
  3. Contour (Đường viền)

Hình 11.4: Mặt nạ phân vùng (Ground truth)

Hàm Load Ảnh và Mặt nạ

Định nghĩa hàm đọc file từ ổ đĩa và chuyển thành Numpy array. Mặt nạ được xử lý thành thang xám (grayscale) và trừ 1 để đưa các nhãn về $[0, 1, 2]$.

```
def path_to_input_image(path):
    return img_to_array(load_img(path, target_size=img_size))

def path_to_target(path):
    img = img_to_array(
        load_img(path, target_size=img_size, color_mode="grayscale")
    )
    # Subtracts 1 so that our labels become 0, 1, and 2
    img = img.astype("uint8") - 1
    return img
```

Nạp Hình ảnh vào mảng NumPy

Khởi tạo hai mảng kích thước cực lớn chứa toàn bộ Data. Ảnh có 3 kênh màu, Mặt nạ chỉ có 1 kênh số nguyên.

```
import numpy as np
import random
from keras.utils import load_img, img_to_array

img_size = (200, 200)
num_imgs = len(input_img_paths)

# Shuffles the file paths
random.Random(1337).shuffle(input_img_paths)
random.Random(1337).shuffle(target_paths)

input_imgs = np.zeros((num_imgs,) + img_size + (3,), dtype="float32")
targets = np.zeros((num_imgs,) + img_size + (1,), dtype="uint8")

# Loops over all elements to fill numpy arrays...
```

Chia tập Huấn luyện và Xác thực

Tương tự bài toán phân loại ảnh, ta cần chia dữ liệu để tránh Overfitting.

```
# Reserves 1,000 samples for validation
num_val_samples = 1000

# Splits the data into a training and a validation set
train_input_imgs = input_imgs[:-num_val_samples]
train_targets = targets[:-num_val_samples]

val_input_imgs = input_imgs[-num_val_samples:]
val_targets = targets[-num_val_samples:]
```

Dù kích thước tập dữ liệu tương đối nhỏ (hơn 7000 ảnh), nhưng nhờ kỹ thuật Data Augmentation và kiến trúc CNN mạnh, ta vẫn có thể học tốt.

Kiến trúc Encoder - Decoder (U-Net style)

Mạng nơ-ron phân vùng có hình phễu, chia làm 2 nửa:

- **Encoder (Mã hóa):** Dùng `Conv2D` với `strides=2` để **Hạ độ phân giải** ảnh xuống (Ví dụ: 200 $\rightarrow$ 100 $\rightarrow$ 50). Nhằm trích xuất thông tin khái niệm (What).
- **Decoder (Giải mã):** Dùng `Conv2DTranspose` với `strides=2` để **Tăng độ phân giải** ảnh lên lại (Ví dụ: 50 $\rightarrow$ 100 $\rightarrow$ 200). Nhằm định vị lại không gian gốc (Where).

Toán học của Transposed Convolution

- Để tăng độ phân giải (Upsampling) mà vẫn duy trì khả năng học hỏi (Learnable), ta dùng **Tích chập Chuyển vị (Transposed Convolution)**.
- Nó hoạt động ngược lại với Tích chập thông thường:
  - Nhận 1 giá trị đầu vào (Pixel).
  - Nhân giá trị đó với một Bộ lọc (Kernel).
  - Phát tán (Broadcast) kết quả ra một vùng không gian lớn hơn ở đầu ra.
- Nhờ đó, mạng có thể học cách "điền" thông tin chi tiết vào những vùng ảnh đã bị thu nhỏ trước đó.

Định nghĩa Nửa thứ nhất - Encoder

Hạ độ phân giải ảnh để nén đặc trưng. Lưu ý ta dùng `strides=2` thay cho MaxPooling để giữ lại thông tin tọa độ không gian (Location info).

```
import keras
from keras.layers import Rescaling, Conv2D, Conv2DTranspose

def get_model(img_size, num_classes):
    inputs = keras.Input(shape=img_size + (3,))
    x = Rescaling(1.0 / 255)(inputs)

    # Encoder part: Downsampling
    x = Conv2D(64, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2D(64, 3, activation="relu", padding="same")(x)
    x = Conv2D(128, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2D(128, 3, activation="relu", padding="same")(x)
    x = Conv2D(256, 3, strides=2, padding="same", activation="relu")(x)
    x = Conv2D(256, 3, activation="relu", padding="same")(x)
```

Định nghĩa Nửa thứ hai - Decoder

Sử dụng `Conv2DTranspose` khôi phục lại độ phân giải 200×200 ban đầu. Output cuối cùng có 3 kênh (tương ứng 3 class) với Softmax.

```
    # Decoder part: Upsampling
    x = Conv2DTranspose(256, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(256, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2DTranspose(128, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(128, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2DTranspose(64, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(64, 3, strides=2, activation="relu", padding="same")(x)

    # Ends with per-pixel softmax classifier
    outputs = Conv2D(num_classes, 3, activation="softmax", padding="same")(x)
    return keras.Model(inputs, outputs)

model = get_model(img_size=img_size, num_classes=3)
```

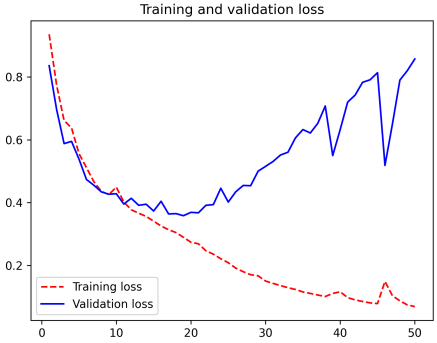
Độ đo IoU (Intersection over Union)

IoU đo phần trăm giao nhau giữa Mặt nạ thực tế và Mặt nạ dự đoán. IoU = Giao / Hợp (Nằm trong khoảng 0-1).

```
foreground_iou = keras.metrics.IoU(
    # Specifies the total number of classes
    num_classes=3,
    # Specifies the class to compute IoU for (0 = foreground)
    target_class_ids=(0,),
    name="foreground_iou",
    # Our targets are sparse (integer class IDs).
    sparse_y_true=True,
    # But our model predictions are a dense softmax!
    sparse_y_pred=False,
)
```

Biểu đồ Loss trong Huấn luyện

- Vì đây là phân loại 3 nhãn cho từng pixel, ta dùng Hàm mất mát: `sparse_categorical_crossentropy`.
- Mô hình hội tụ rất tốt ở giai đoạn đầu.
- Bắt đầu xuất hiện Overfitting sau epoch thứ 25.



Hình 11.5: Quá trình Training Loss

Biên dịch và Huấn luyện (Train)

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=[foreground_iou],
)

callbacks = [
    keras.callbacks.ModelCheckpoint(
        "oxford_segmentation.keras", save_best_only=True
    )
]

history = model.fit(
    train_input_imgs, train_targets,
    epochs=50, batch_size=64,
    callbacks=callbacks,
    validation_data=(val_input_imgs, val_targets),
)
```

Kết quả Dự đoán (Prediction)

- Trực quan hóa kết quả dự đoán của mô hình tự thiết kế.
- Mô hình bắt được hình dáng của con mèo khá tốt (Khu vực Tiền cảnh).
- Tuy nhiên, các chi tiết nhỏ ở đường viền (như tai, râu) vẫn còn nhiễu và bị mờ (Artifacts).
- Để đạt độ chính xác SOTA, ta cần các mô hình lớn hơn rất nhiều.

Hình 11.6: Mặt nạ do mô hình dự đoán

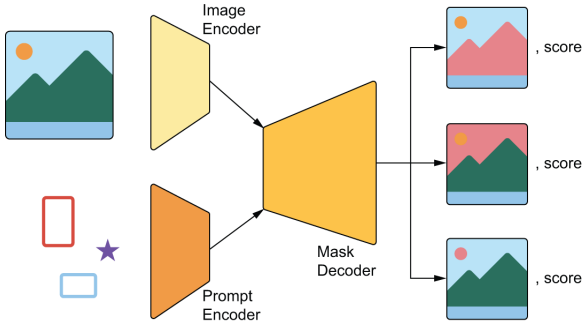
Kỷ nguyên của Foundation Models trong CV

- Natural Language Processing (NLP) đã thay đổi mãi mãi với sự xuất hiện của các Foundation Models (Như GPT-3, GPT-4) - Một mô hình huấn luyện một lần, giải quyết được mọi tác vụ (Zero-shot).
- Vào tháng 4/2023, Meta AI công bố **Segment Anything Model (SAM)**, đánh dấu cột mốc Foundation Model đầu tiên của ngành Thị giác Máy tính (Computer Vision).
- SAM có khả năng phân vùng "Bất cứ thứ gì" trong ảnh, mà không cần huấn luyện lại.

Kiến trúc của SAM

SAM gồm 3 thành phần cốt lõi:

1. **Image Encoder:** Vision Transformer (ViT) kích thước cực lớn, giải nén toàn bộ thông tin ảnh.
2. **Prompt Encoder:** Biểu diễn các Lệnh nhắc của người dùng (Điểm, Hộp, Text).
3. **Mask Decoder:** Một mạng nhẹ hợp nhất thông tin từ Image và Prompt để xuất ra Mặt nạ (Mask).



Hình 11.7: Kiến trúc tổng thể của Segment Anything

Siêu Dữ liệu SA-1B

- Sức mạnh của SAM đến từ Dữ liệu. Meta AI đã xây dựng tập dữ liệu **SA-1B (Segment Anything 1-Billion)**.
- Khối lượng khổng lồ: **11 triệu hình ảnh** với hơn **1.1 Tỷ mặt nạ phân vùng** chất lượng cao (gấp 400 lần các tập dữ liệu trước đó).
- Nhờ vậy, SAM có khả năng phân vùng Zero-shot với mọi thể loại vật thể từ sinh học, phương tiện, đến y tế.

Hình 11.8: Mật độ mặt nạ trong tập dữ liệu SA-1B

Khởi tạo SAM từ KerasHub

Mọi thứ đều có sẵn trong KerasHub, ta gọi mô hình siêu lớn với Preset `sam_huge_sa1b`.

```
import keras_hub

# Instantiates SAM and downloads its weights
model = keras_hub.models.ImageSegmenter.from_preset("sam_huge_sa1b")

print(model.count_params())
# Output: 641090864
```

Với 641 triệu tham số, đây là mô hình lớn nhất mà chúng ta sử dụng! Tốc độ tiến hóa của Pre-trained Models tỷ lệ thuận với lượng dữ liệu cung cấp (Sẽ học ở Chương 16).

Môi trường Thử nghiệm

- Ta sẽ thử nghiệm sức mạnh của SAM trên một bức ảnh thực tế chứa rất nhiều loại Trái cây (Chuối, Đào, Xoài).
- Bức ảnh này có cấu trúc phức tạp với vật thể chồng chéo và bóng đổ.
- Mục tiêu: Gọi API của SAM để phân vùng chúng thông qua các kiểu **Prompting (Lệnh nhắc)** khác nhau.

Hình 11.9: Ảnh Trái cây đầu vào

Tiền xử lý Ảnh cho SAM (Padding)

SAM yêu cầu ảnh đầu vào có kích thước cố định 1024×1024. Ta không được bóp méo tỷ lệ khung hình, nên phải dùng Padding.

```
from keras import ops

image_size = (1024, 1024)

def resize_and_pad(x):
    # Resizes the image to 1024 on its longest side,
    # then pads the remaining pixels with 0 to keep aspect ratio
    return ops.image.resize(x, image_size, pad_to_aspect_ratio=True)

image = resize_and_pad(image_array)
```

Hàm tiện ích (Utilities)

Chúng ta viết các hàm hiển thị ảnh với Matplotlib để vẽ Bounding Box và Points.

```python
import matplotlib.pyplot as plt
from keras import ops

def show_image(image, ax):
    ax.imshow(ops.convert_to_numpy(image).astype("uint8"))

def show_points(points, ax):
    x, y = points[:, 0], points[:, 1]
    ax.scatter(x, y, c="green", marker="*", s=375, ec="white", lw=1.25)

def show_box(box, ax):
    box = box.reshape(-1)
    x0, y0 = box[0], box[1]
    w, h = box[2] - box[0], box[3] - box[1]
    ax.add_patch(plt.Rectangle((x0, y0), w, h, ec="red", fc="none", lw=2))
```

Hàm Trích xuất Mặt nạ (Get Mask)

Mỗi lần được Prompt, SAM sẽ trả về 4 Mặt nạ ứng viên dự đoán tốt nhất, xếp hạng theo điểm chất lượng.

```
def get_mask(sam_outputs, index=0):
    # Select the mask candidate with highest score (index 0)
    mask = sam_outputs["masks"][0][index]
    mask = np.expand_dims(mask, axis=-1)
    # Resize mask back to fit the image
    mask = resize_and_pad(mask)
    # Convert logits into a boolean mask
    return ops.convert_to_numpy(mask) > 0.0
```

Hàm này sẽ trích xuất ra một ma trận Boolean (True/False) chỉ định các Pixel thuộc khu vực Tiền cảnh.

Mặt nạ thay thế (Alternative Masks)

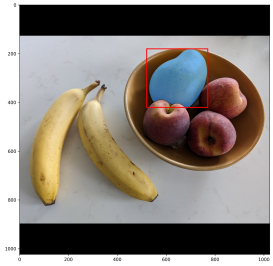
- Khi Prompt một điểm nhập nhằng (Ví dụ: Giữa nải chuối).
- SAM sẽ nội suy và sinh ra 3 phương án dự phòng (Alternative Masks) với nhiều cấp độ bao phủ khác nhau.
- Điểm thú vị là: Một phương án sẽ bắt riêng lẻ 1 trái chuối, phương án khác bao trùm cả nải chuối! (Rất thông minh).

Hình 11.10: Các Mặt nạ thay thế của SAM

Nhắc bằng Hộp giới hạn (Box Prompting)

Cung cấp 1 hộp bao phủ trái xoài (Tọa độ Góc trên-trái và Góc dưới-phải).

```
input_box = np.array([
    [520, 180], # Top-left corner
    [770, 420], # Bottom-right corner
])

outputs = model.predict(
    {
        "images": ops.expand_dims(img,
    axis=0),
        "boxes": ops.expand_dims(input_box
    , axis=(0, 1)),
    }
)
```

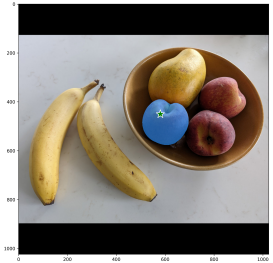


Hình 11.11: SAM lập tức trả về Mặt nạ Xoài chính xác

Nhắc bằng Điểm đơn (Single Point Prompting)

Nhập đúng 1 tọa độ rơi vào quả Đào. (Nhãn 1 nghĩa là Tiền cảnh, chọn cái này!)

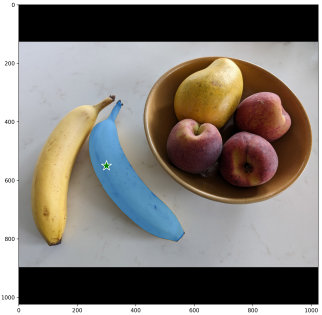
```
# Coordinates of our point
input_point = np.array([[580, 450]])

# 1 means foreground, 0 means background
input_label = np.array([1])

outputs = model.predict(
    {
        "images": ops.expand_dims(image,
    axis=0),
        "points": ops.expand_dims(
    input_point, axis=0),
        "labels": ops.expand_dims(
    input_label, axis=0),
    }
)
```



Hình 11.12: SAM lan truyền điểm đó thành Mặt nạ Đào

Sức mạnh nội suy của Point Prompt

- Tương tự, nếu ta truyền Tọa độ điểm của một quả Chuối bị khuất.
- Kể cả khi quả chuối bị che bởi bóng đổ hay trái cây khác, công cụ ViT khổng lồ của SAM vẫn nội suy (Interpolate) ra đường viền hoàn mỹ.
- Quá trình này diễn ra theo thời gian thực (Real-time) trong trình duyệt nhờ Mask Decoder siêu nhẹ.



Hình 11.13: Phân vùng Chuối bằng 1 Point Prompt

Tổng kết Chương 11

- **Image Segmentation** là bài toán tìm ranh giới chính xác đến từng pixel cho vật thể (Semantic, Instance, Panoptic).
- **U-Net & Transposed Convolution:** Một mô hình kinh điển giải quyết phân vùng bằng kiến trúc Encoder (hạ phân giải để tìm Đặc trưng) và Decoder (tăng phân giải để Phục hồi không gian).
- **Segment Anything (SAM):** Đưa Computer Vision bước vào kỷ nguyên Foundation Model, cho phép phân vùng Zero-shot "Mọi thứ" chỉ bằng các gợi ý đơn giản (Point, Box, Text).